A3-PS008-TC521

Network Management Agent for Reviewed Configuration Changes

Negative Latency
Sruja Brahmbhatt · Mana Fatnani · Dr. Kaushal Shah

AIORI Hackathon 3 · Category: Cybersecurity
October 2026

Problem Description

Typed by hand. Engineers change live routers by typing CLI commands. A wrong digit in a prefix or next hop is accepted without complaint.

Checked by no one. Each command can be valid on its own and still conflict with what another engineer configured elsewhere.

Hard to undo. There is no test before the change and no reliable rollback if it fails halfway.

```
# engineer A, on R1
r1(config)# ip route 172.16.50.0/24 10.0.12.2

# engineer B, on R2, a week later
r2(config)# ip route 172.16.50.0/24 10.0.12.1

# both commands accepted
```

Two valid commands. Together they bounce partner traffic R1 → R2 → R1 until it expires. Nothing in the CLI warns either engineer.

Background

Static route
A route written by hand: “send 172.16.50.0/24 to R2.” It never updates itself.

OSPF
Routers advertise their links and costs, then each computes shortest paths with Dijkstra. Adapts when links fail.

Routing loop
Routers forward a packet in a circle until its TTL runs out. One bad static route is enough.

Intent-based networking
The operator states the goal; the system derives the device commands and checks them.

Digital twin
A software model of the network used to test a change before it reaches real devices.

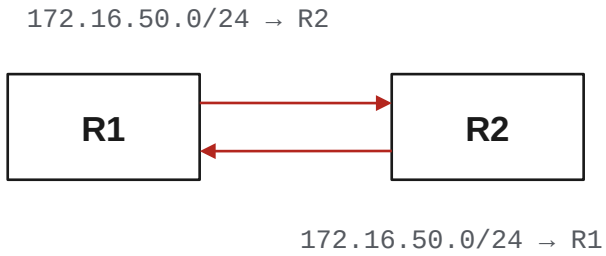


Figure: a two-router routing loop. Each router believes the other is the way out.

Literature & RFC Review

Source	Key idea	How our design uses it
RFC 6241, NETCONF	Candidate datastore; validate, commit and confirmed-commit	Candidate config is validated in the twin, committed, and reverted on failure
RFC 7950, YANG 1.1	Typed, structured models of device configuration	Structured router config and a typed intent schema instead of raw CLI text
RFC 2328, OSPFv2	Link-state routing and shortest-path-first	Twin rebuilds OSPF routes with Dijkstra over link costs
Batfish (NSDI 2015)	Offline analysis of network configurations	Same principle: prove a change is safe in a model first
FRRouting, containerlab	Open-source routing stack; reproducible lab topologies	Single router platform; 3-router testbed

Proposed Solution

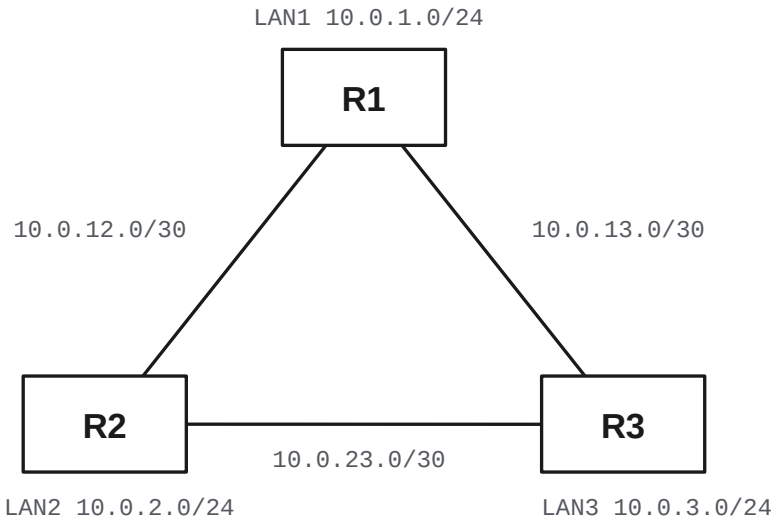


Figure: 3-router FRRouting testbed

1. **Intent schema:** a typed YAML request; two intent kinds, ospf_link and static_route
2. **Generator:** derives FRR commands from a diff of current and target config
3. **Digital twin:** rebuilds every routing table; blocks loops, conflicts and lost reachability
4. **Human approval gate:** nothing deploys until a second person approves
5. **Deploy and verify:** snapshot all routers, apply, re-read the running config
6. **Rollback:** restore every router from the snapshot if anything fails

Optimization Proposed

Aspect	Manual CLI (before)	With our agent (after)
Input	Free-form commands	Typed schema; 2 intent kinds
Allowed commands	Anything, including write erase	7-pattern allowlist, checked twice
Testing before change	None	Digital twin: loops, conflicts, regressions
Approval	Informal or none	Mandatory gate; requester cannot self-approve
Network changed after review	Goes unnoticed	Config fingerprint check marks it STALE
Failure halfway through	Half-configured network	All 3 routers rolled back together
Device silently ignores change	Goes unnoticed	Post-deploy verification triggers rollback
Change history	None	Hash-chained, tamper-evident audit log

Solution Architecture & Design

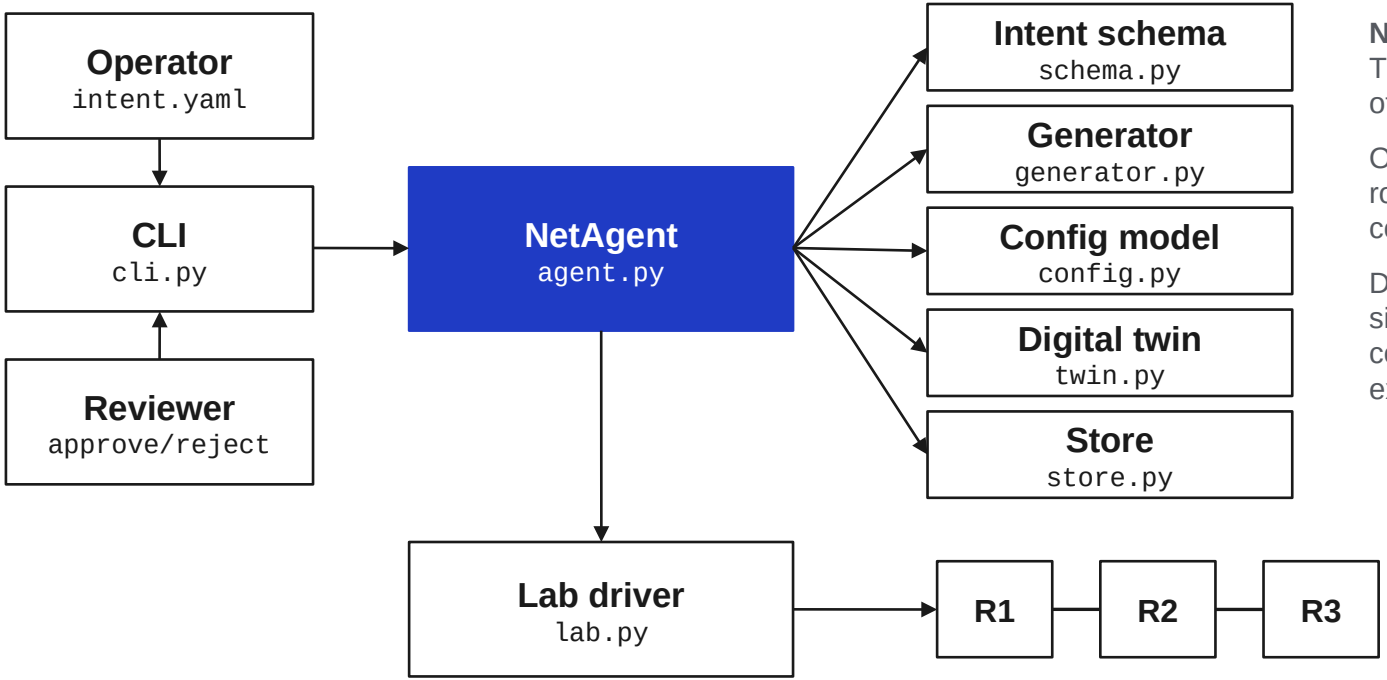


Figure: System Architecture UML Diagram

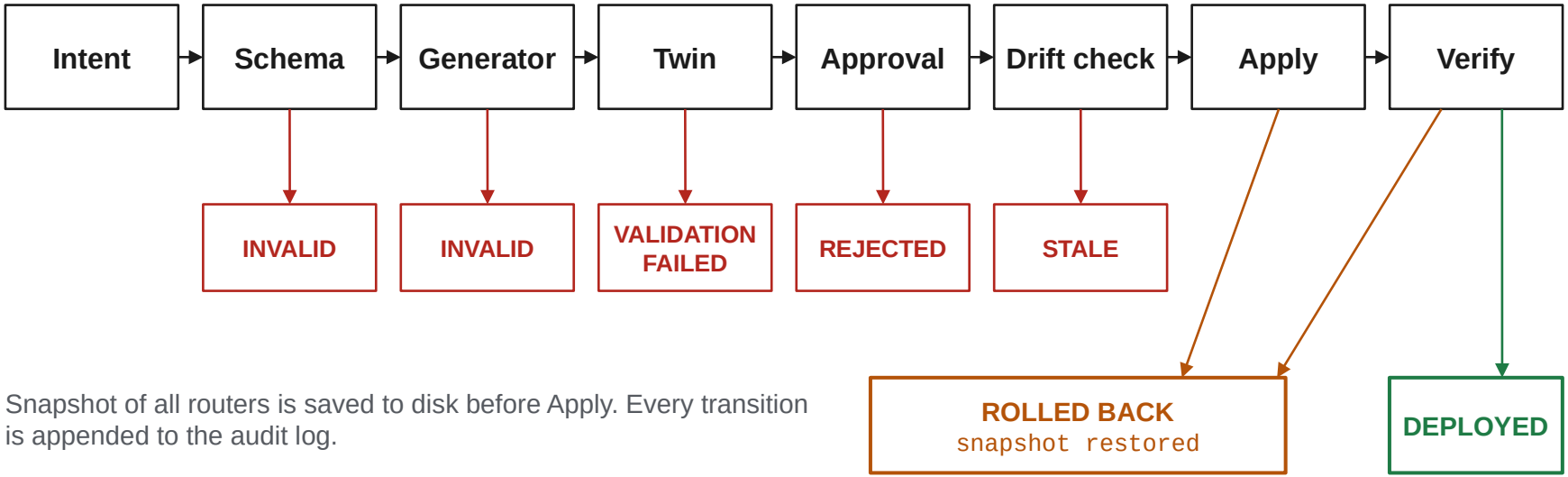
Notes

The agent is the only caller of every module.

Only the lab driver touches routers, and it re-checks the command allowlist.

Driver is swappable: simulated FRR, or real FRR containers over docker exec.

Data Flow Diagram



Snapshot of all routers is saved to disk before Apply. Every transition is appended to the audit log.

Figure: Data Flow Diagram (DFD)

Deployment Timeline

Milestone	Status
Pipeline prototype on simulated FRR lab	**Done**
Test suite: conflicts, invalid configs, rejections, failed deploys	**Done**
Adapt to the 2 mentor-defined intents	Planned
Real FRR testbed with containerlab; ping checks	Planned
Technical report and final demonstration	Planned

```
$ python -m pytest -q tests
.........................
27 passed in 0.25s

$ python -m netagent audit
22 entries, chain intact
```

Methodology: every safety rule is backed by an automated test scenario before it is demonstrated.

Bibliography

[1] R. Enns, M. Bjorklund, J. Schoenwaelder, A. Bierman. “Network Configuration Protocol (NETCONF).” RFC 6241, IETF, 2011.

[2] M. Bjorklund. “The YANG 1.1 Data Modeling Language.” RFC 7950, IETF, 2016.

[3] J. Moy. “OSPF Version 2.” RFC 2328, IETF, 1998.

[4] A. Fogel et al. “A General Approach to Network Configuration Analysis.” USENIX NSDI, 2015.

[5] FRRouting Project. FRR User Documentation. docs.frrouting.org

[6] containerlab. Network labs as code. containerlab.dev

Thanks for the ACK

No packets were looped in the making of this presentation.

Questions? · Negative Latency · A3-PS008-TC521